

Table of Contents

Comprehensive Question Bank: Unit 4 — Development of Graphical User Interface

(GUI).....	2
Section A: Multiple-Choice Questions (MCQs).....	2
Topic 4.1.1: What is a GUI? Why is it Important?	2
Topic 4.1.2: Overview of Tkinter as Python's Built-in GUI Toolkit	3
Topic 4.1.3: Tkinter Components and Widgets	5
Topic 4.1.4: Layout Management (pack(), grid(), place()).....	6
Topic 4.1.5: Event Handling and Interactivity	8
Topic 4.2.1: Introduction to Databases	10
Topic 4.2.2: Overview of Relational Databases and SQL Structure	11
Topic 4.2.3: Connecting Python to a Database	13
Topic 4.2.4: CRUD Operations in Python GUI Applications.....	15
Section B: Short Answer Questions (SQs)	17
Topic 4.1.1: What is a GUI? Why is it Important?	17
Topic 4.1.2: Overview of Tkinter as Python's Built-in GUI Toolkit	17
Topic 4.1.3: Tkinter Components and Widgets	18
Topic 4.1.4: Layout Management	18
Topic 4.1.5: Event Handling and Interactivity	18
Topic 4.2.1: Introduction to Databases	19
Topic 4.2.2: Overview of Relational Databases and SQL Structure	19
Topic 4.2.3: Connecting Python to a Database	19
Topic 4.2.4: CRUD Operations in Python GUI Applications.....	20
Section C: Long / Extensive Questions (LQs)	20
Topic 4.1: Graphical User Interface (GUI) Development with Tkinter	20
Topic 4.2: Working with Databases in Python.....	21

Comprehensive Question Bank: Unit 4 — Development of Graphical User Interface (GUI)

Section A: Multiple-Choice Questions (MCQs)

Topic 4.1.1: What is a GUI? Why is it Important?

1. GUI stands for:
 - A) General User Interaction
 - B) Graphical User Interface**
 - C) Guided User Instruction
 - D) General Utility Interface
2. Which of the following best defines a Graphical User Interface?
 - A) A programming language used to write scripts
 - B) A visual way for users to interact with software using elements like buttons, menus, and forms**
 - C) A type of database used to store images
 - D) A command typed into a terminal window
3. A Command-Line Interface (CLI) differs from a GUI mainly because a CLI:
 - A) Uses buttons and icons for interaction
 - B) Requires the user to type exact commands**
 - C) Is only used on smartphones
 - D) Cannot be used to run Python programs
4. Which of the following is NOT a typical benefit of using a GUI over a CLI?
 - A) Easier for non-technical users
 - B) Provides visual feedback
 - C) Requires memorizing exact command syntax**
 - D) Reduces the need to remember complex commands
5. GUI applications are commonly used in:
 - A) Only desktop software
 - B) Only mobile apps
 - C) Only web applications
 - D) Desktop software, web applications, and mobile apps**

6. In the 1970s, the research lab credited with pioneering the GUI concept (windows, icons, menus, pointer) was:
- A) IBM Research
 - B) Xerox PARC**
 - C) Bell Labs
 - D) MIT Media Lab
7. The acronym WIMP, associated with early graphical interfaces, stands for:
- A) Windows, Input, Mouse, Pixels
 - B) Windows, Icons, Menus, Pointer**
 - C) Widgets, Icons, Mainloop, Program
 - D) Windows, Interface, Menu, Program
8. Which entrepreneur famously visited Xerox PARC in 1979 and later brought GUI concepts to the Macintosh?
- A) Bill Gates
 - B) Steve Jobs**
 - C) Alan Turing
 - D) Edgar Codd
9. Compared to a CLI, a GUI generally improves usability by:
- A) Forcing the user to memorize more commands
 - B) Providing visual feedback and reducing the need for memorized commands**
 - C) Removing the need for any user input
 - D) Making the software run faster
-

Topic 4.1.2: Overview of Tkinter as Python's Built-in GUI Toolkit

1. Tkinter is best described as:
- A) A third-party library that must be downloaded separately
 - B) Python's standard, built-in library for creating GUI applications**
 - C) A database management system
 - D) A web development framework
2. Which of the following is an advantage of Tkinter?
- A) It requires a paid license
 - B) It only works on Windows
 - C) It comes pre-installed with Python**
 - D) It cannot create buttons or labels

3. Tkinter is most suitable for building:
- A) Massive enterprise-scale distributed systems
 - B) Small-to-medium sized GUI applications**
 - C) Only mobile applications
 - D) Operating system kernels
4. Which function call is used to create the main application window in Tkinter?
- A) `tk.Window()`
 - B) `tk.Tk()`**
 - C) `tk.MainWindow()`
 - D) `tk.Frame()`
5. Which method must be called to keep a Tkinter window open and responsive to user actions?
- A) `window.run()`
 - B) `window.start()`
 - C) `window.mainloop()`**
 - D) `window.show()`
6. What is the correct order of steps to build a basic Tkinter GUI application?
- A) Start mainloop → Create window → Add widgets
 - B) Create window → Define functions/widgets → Place widgets → Start mainloop**
 - C) Add widgets → Start mainloop → Create window
 - D) Define functions → Start mainloop → Create window
7. If a Tkinter script does not call `mainloop()`, what will typically happen when the script runs?
- A) The window will stay open forever
 - B) The window will open but all buttons will be disabled
 - C) The script will run and exit almost immediately without displaying an interactive window**
 - D) Python will raise a syntax error
8. `window.title("Simple GUI Example")` is used to:
- A) Create a new button labeled "Simple GUI Example"
 - B) Set the text shown in the window's title bar**
 - C) Save the window as a file named "Simple GUI Example"
 - D) Change the color of the window
9. `window.geometry("300x200")` sets:
- A) The database size
 - B) The width and height of the window in pixels**
 - C) The number of widgets allowed
 - D) The font size of all text
-

Topic 4.1.3: Tkinter Components and Widgets

1. In Tkinter, a "widget" refers to:
 - A) A type of database table
 - B) A visual GUI element such as a label, button, entry field, menu, or listbox**
 - C) A Python data type
 - D) A layout manager only
2. The root window created by `tk.Tk()` is best described as:
 - A) A widget used only for buttons
 - B) The main container that holds all other GUI elements**
 - C) A database connection object
 - D) An event handler function
3. A Frame in Tkinter is primarily used to:
 - A) Display images only
 - B) Group and organize related widgets inside a window**
 - C) Connect Python to a database
 - D) Start the event loop
4. Which widget is used to display text or an image that the user cannot edit?
 - A) Entry
 - B) Label**
 - C) Listbox
 - D) Menu
5. Which widget allows the user to type in a single line of text, such as a username?
 - A) Label
 - B) Listbox
 - C) Entry**
 - D) Frame
6. Which widget is best suited for showing a scrollable list of selectable items, such as a list of songs?
 - A) Entry
 - B) Label
 - C) Listbox**
 - D) Button
7. Which widget provides a dropdown list of options, such as File > Open, at the top of a window?
 - A) Frame
 - B) Listbox
 - C) Menu**
 - D) Entry

8. Which widget is used to trigger an action, such as submitting a form, when clicked?
- A) Label
 - B) Button**
 - C) Entry
 - D) Frame
9. In the example code `entry.pack(pady=10)`, what does `pady` control?
- A) The width of the entry box
 - B) The vertical padding (space) around the widget**
 - C) The password masking character
 - D) The number of characters allowed
10. Two Frames, `top_frame` and `bottom_frame`, are created inside the same window mainly to:
- A) Create two separate databases
 - B) Divide the window into organized sections for grouping widgets**
 - C) Run two event loops simultaneously
 - D) Automatically resize the window
11. Which statement about widgets and frames is TRUE?
- A) A Frame can only contain one widget
 - B) Widgets can be placed inside a Frame, and Frames are placed inside the main window**
 - C) A Label can hold other widgets like a Frame does
 - D) A Listbox cannot display more than one item
-

Topic 4.1.4: Layout Management (`pack()`, `grid()`, `place()`)

1. The purpose of a layout manager in Tkinter is to:
- A) Connect the GUI to a database
 - B) Decide the position and size of each widget on the screen**
 - C) Handle button click events
 - D) Create new windows automatically
2. The `pack()` layout manager arranges widgets by:
- A) Using row and column coordinates
 - B) Stacking them vertically or horizontally in the order they are packed**
 - C) Using exact x and y pixel coordinates
 - D) Randomly placing them on the screen

3. The `grid()` layout manager arranges widgets:
- A) In a single vertical stack
 - B) In rows and columns, like a table**
 - C) At exact pixel coordinates only
 - D) In a circular pattern
4. The `place()` layout manager arranges widgets by:
- A) Automatically centering them
 - B) Positioning them at exact (x, y) pixel coordinates**
 - C) Stacking them in a list
 - D) Aligning them into rows and columns only
5. Which layout manager would be most appropriate for building a structured registration form with labeled fields in neat rows?
- A) `pack()`
 - B) `grid()`**
 - C) `place()`
 - D) `Frame()`
6. In `button2.grid(row=0, column=1, padx=10, pady=20)`, the parameters `padx` and `pady` control:
- A) The row and column position
 - B) The horizontal and vertical spacing around the widget**
 - C) The widget's width and height
 - D) The widget's background color
7. What does `columnspan=2` do when used with `grid()`?
- A) Splits a widget into two rows
 - B) Makes a widget span across two columns**
 - C) Doubles the widget's height
 - D) Creates two identical widgets
8. Which of the following is a known risk of using `place()` for layout?
- A) It cannot be used with buttons
 - B) It uses exact coordinates and can break easily when the window is resized**
 - C) It automatically creates a database
 - D) It cannot be combined with `pady`
9. What happens if you try to use `pack()` and `grid()` on different widgets within the SAME parent container?
- A) Tkinter automatically converts one to match the other
 - B) Tkinter raises an error or produces unpredictable layout behavior**
 - C) Both layout managers work together seamlessly
 - D) Only the widget added last will be displayed

10. `window.grid_columnconfigure(0, weight=1)` is primarily used to:
- A) Delete column 0 from the grid
 - B) Allow column 0 to stretch and resize responsively when the window is resized**
 - C) Set the background color of column 0
 - D) Assign a primary key to column 0
11. A "responsive" interface is one that:
- A) Responds only to keyboard input
 - B) Adjusts its layout appropriately when the window is resized**
 - C) Cannot be resized at all
 - D) Uses only the `place()` layout manager
12. Designing a "clean" interface primarily involves:
- A) Using as many colors as possible
 - B) Providing enough spacing, clear text, and proper alignment between widgets**
 - C) Avoiding the use of Labels
 - D) Placing all widgets at coordinate (0,0)
-

Topic 4.1.5: Event Handling and Interactivity

1. In event-driven programming, the program primarily:
- A) Executes all code from top to bottom without stopping
 - B) Waits idly until a specific event occurs, then responds**
 - C) Only runs once and then closes immediately
 - D) Cannot respond to user actions
2. An "event" in the context of Tkinter refers to:
- A) A type of widget
 - B) An action performed by the user, such as a click or key press**
 - C) A method used to connect to a database
 - D) A layout manager
3. The `mainloop()` function is responsible for:
- A) Connecting Python to a database
 - B) Continuously checking for events while the window remains open**
 - C) Formatting SQL queries
 - D) Creating new widgets automatically
4. In Tkinter, the `command=` parameter is most commonly used with which widget?
- A) Label
 - B) Button**
 - C) Frame
 - D) Listbox

5. What is the correct way to link a function named `on_click` to a button using `command=`?
- A) `command=on_click()`
 - B) `command=on_click`**
 - C) `command="on_click"`
 - D) `command=call(on_click)`
6. Why is it incorrect to write `command=my_function()` instead of `command=my_function`?
- A) Tkinter does not allow function names longer than 10 characters
 - B) Using parentheses calls the function immediately instead of when the button is clicked**
 - C) `command=` only accepts strings
 - D) It will cause the window to not open at all
7. The `.bind()` method in Tkinter is generally used to:
- A) Connect Python to a database
 - B) Attach a function to a broader range of events, such as key presses or mouse movement, on any widget**
 - C) Bind two windows together into one
 - D) Set the width of a widget
8. What does `entry.get()` do in a Tkinter program?
- A) Deletes the text inside the entry field
 - B) Retrieves the current text typed into the entry field**
 - C) Creates a new entry field
 - D) Connects the entry field to a database
9. If a heavy 30-second loop is placed directly inside a button's callback function, what is the most likely result?
- A) The loop will run in a separate thread automatically
 - B) The GUI window will freeze and appear as "Not Responding" until the loop finishes**
 - C) The button will disable itself automatically
 - D) Tkinter will skip the loop entirely
10. In the login form example, `messagebox.showerror("Error", "Invalid Username or Password")` is used to:
- A) Automatically close the entire program
 - B) Display a popup dialog indicating an error occurred**
 - C) Delete the incorrect entry from the database
 - D) Restart the event loop

11. Using `show="*"` as a parameter of an `Entry` widget is used to:
- A) Make the entry field larger
 - B) Hide typed characters, such as for password fields**
 - C) Automatically validate the input
 - D) Connect the entry to a database
12. Which of these best summarizes event-driven programming, as illustrated by the "waiter and bell" analogy?
- A) The waiter cooks food continuously regardless of customer actions
 - B) The waiter waits idly and only acts when a specific event (the bell) occurs**
 - C) The waiter takes orders from every table at once, regardless of events
 - D) The waiter has no connection to customer actions
-

Topic 4.2.1: Introduction to Databases

1. A database is best defined as:
- A) A single unformatted text file
 - B) An organized collection of data stored in a structured format**
 - C) A type of GUI widget
 - D) A programming language
2. Which of the following is a key benefit of using a database over a plain flat file?
- A) Flat files are always faster to search
 - B) Databases reduce data duplication and improve reliability**
 - C) Databases cannot store text data
 - D) Flat files enforce stricter data structure
3. In database terminology, an "entity" refers to:
- A) A column in a table
 - B) A real-world object, person, place, event, or concept about which data is stored**
 - C) A type of SQL command
 - D) A GUI widget
4. In database terminology, an "attribute" refers to:
- A) A relationship between two tables
 - B) A property or characteristic of an entity**
 - C) A unique record identifier only
 - D) The name of the database file

5. For the entity "Student," which of the following is an example of an attribute?
- A) Enrolls in
 - B) Roll Number**
 - C) Course
 - D) Teacher
6. A "relationship" in database terminology describes:
- A) A single column's data type
 - B) How two or more entities are connected with each other**
 - C) The physical location of the database file
 - D) The GUI layout manager used
7. An "identifier" in a database is best described as:
- A) Any column containing text
 - B) An attribute that uniquely identifies each record of an entity**
 - C) A widget used to display records
 - D) A type of layout manager
8. Which of the following would most likely serve as an identifier for the entity "Student"?
- A) Class
 - B) Roll Number**
 - C) Age
 - D) Student Name
9. IBM mathematician Edgar F. Codd is historically credited with:
- A) Inventing the GUI
 - B) Inventing the relational database model in 1970**
 - C) Discovering SQL injection
 - D) Creating the Tkinter library
10. Before the relational database model, data was often stored on:
- A) SQLite files
 - B) Physical magnetic tapes in inconsistent formats**
 - C) Cloud servers
 - D) Tkinter widgets
-

Topic 4.2.2: Overview of Relational Databases and SQL Structure

1. A relational database organizes data primarily using:
- A) A single unstructured document
 - B) Tables made of rows and columns**
 - C) Only GUI widgets
 - D) Randomly ordered lists

2. SQL stands for:
- A) Structured Question Language
 - B) Structured Query Language**
 - C) Sequential Query Logic
 - D) System Query List
3. In a database table, a "row" is also referred to as a:
- A) Field
 - B) Record**
 - C) Column
 - D) Attribute
4. In a database table, a "column" is also referred to as a:
- A) Record
 - B) Field**
 - C) Tuple
 - D) Primary Key
5. A Primary Key in a database table is used to:
- A) Link two tables together only
 - B) Uniquely identify each record in a table**
 - C) Store only text-based data
 - D) Format the GUI layout
6. A Foreign Key is best described as:
- A) A key used only for encryption
 - B) A field that links one table to another by referring to the primary key of another table**
 - C) A column that must always contain duplicate values
 - D) A widget used to display Foreign records
7. Given a **Student** table with a **DeptID** column that refers to the **DeptID** column of a separate **Department** table, **DeptID** in the **Student** table is functioning as a:
- A) Primary Key
 - B) Foreign Key**
 - C) Identifier only
 - D) Attribute with no relationship
8. Which SQL command category is used to interact with and manage data in a relational database?
- A) HTML
 - B) SQL**
 - C) CSS
 - D) Tkinter

9. Why can a column like `DeptID` (appearing multiple times for different students) NOT serve as the Primary Key of the `Student` table?
- A) Because Primary Keys must always be text
 - B) Because a Primary Key must be unique for every row, and `DeptID` repeats across multiple students**
 - C) Because Foreign Keys are always numeric
 - D) Because Primary Keys cannot be integers
-

Topic 4.2.3: Connecting Python to a Database

1. Which Python library is built-in and commonly used to work with small, file-based databases?
 - A) `tkinter`
 - B) `sqlite3`**
 - C) `pandas`
 - D) `matplotlib`
2. Compared to `sqlite3`, MySQL is typically used for:
 - A) Smaller and simpler applications only
 - B) Larger and more complex database systems**
 - C) Creating GUI widgets
 - D) Formatting text files
3. In the "bank vault" analogy, opening the database connection with `connect()` is compared to:
 - A) The robot arm reading a shelf
 - B) Opening the vault door**
 - C) Locking the vault permanently
 - D) Deleting all items from the vault
4. In the "bank vault" analogy, the `cursor()` object is compared to:
 - A) The vault door itself
 - B) A robot arm that reads and writes items on the shelves**
 - C) The permanent lock on the vault
 - D) A GUI Button widget
5. What does the following line of code do?

```
connection = sqlite3.connect("school.db")
```

 - A) Deletes the "school.db" file
 - B) Opens a connection to "school.db", creating the file if it does not already exist**
 - C) Creates a new GUI window
 - D) Runs a SELECT query on "school.db"

6. Which method is used to execute a SQL command using a cursor object?
- A) `cursor.run()`
 - B) `cursor.execute()`**
 - C) `cursor.query()`
 - D) `cursor.command()`
7. What is the purpose of `CREATE TABLE IF NOT EXISTS` in SQL?
- A) It always deletes and recreates the table
 - B) It creates a new table only if a table with that name doesn't already exist**
 - C) It permanently deletes the database
 - D) It is used only for reading data
8. After executing `INSERT INTO` statements, which method must be called to permanently save the changes to disk?
- A) `cursor.save()`
 - B) `connection.commit()`**
 - C) `connection.close()`
 - D) `cursor.insert()`
9. What happens to `INSERT`, `UPDATE`, or `DELETE` changes if `connection.commit()` is never called?
- A) They are saved immediately regardless
 - B) They are NOT permanently saved to the database**
 - C) Python raises a fatal error immediately
 - D) They are automatically converted into a GUI widget
10. Which method retrieves all rows returned by a `SELECT` query as a list?
- A) `cursor.getall()`
 - B) `cursor.fetchall()`**
 - C) `cursor.readall()`
 - D) `connection.selectall()`
11. Which line of code correctly closes a database connection?
- A) `cursor.close()`
 - B) `connection.close()`**
 - C) `connection.end()`
 - D) `sqlite3.close()`
12. What is the correct order of operations to safely insert and permanently save data using `sqlite3`?
- A) `commit()` → `connect()` → `execute()` → `close()`

- B) `connect()` → `cursor()` → `execute()` → `commit()` → `close()`
- C) `close()` → `connect()` → `execute()` → `commit()`
- D) `execute()` → `connect()` → `commit()` → `cursor()`
-

Topic 4.2.4: CRUD Operations in Python GUI Applications

1. CRUD stands for:
 - A) Create, Run, Upload, Delete
 - B) Create, Read, Update, Delete**
 - C) Copy, Retrieve, Upload, Download
 - D) Create, Remove, Update, Manage
2. Which SQL command corresponds to the "Create" operation in CRUD?
 - A) `SELECT`
 - B) `INSERT INTO`**
 - C) `UPDATE`
 - D) `DELETE FROM`
3. Which SQL command corresponds to the "Read" operation in CRUD?
 - A) `INSERT INTO`
 - B) `SELECT`**
 - C) `UPDATE`
 - D) `DELETE FROM`
4. Which SQL command corresponds to the "Update" operation in CRUD?
 - A) `SELECT`
 - B) `INSERT INTO`
 - C) `UPDATE`**
 - D) `DELETE FROM`
5. Which SQL command corresponds to the "Delete" operation in CRUD?
 - A) `SELECT`
 - B) `INSERT INTO`
 - C) `UPDATE`
 - D) `DELETE FROM`**
6. The "Read" operation in CRUD is unique among the four operations because it:
 - A) Always deletes old data first
 - B) Does not change the stored data — it only retrieves it**
 - C) Requires a GUI Button widget
 - D) Cannot be performed using SQL

7. A "parameterized query," such as `cursor.execute("INSERT INTO students (name) VALUES (?)", (name,))`, is primarily used to:
- A) Make queries run faster in all cases
 - B) Safely insert user input into SQL, preventing SQL injection**
 - C) Automatically create GUI widgets
 - D) Permanently delete a table
8. In 1998, the researcher known as "Rain Forest Puppy" is historically credited with:
- A) Inventing the relational database model
 - B) Discovering the SQL injection vulnerability**
 - C) Creating the Tkinter library
 - D) Inventing the GUI
9. Which of the following raw SQL statements is VULNERABLE to SQL injection?
- A) `cursor.execute("DELETE FROM students WHERE name = ?", (name,))`
 - B) `cursor.execute("DELETE FROM students WHERE name = '" + name + "'")`**
 - C) `cursor.execute("SELECT * FROM students")`
 - D) `cursor.execute("CREATE TABLE IF NOT EXISTS students (id INTEGER PRIMARY KEY)")`
10. If a user enters `'X' OR '1'='1` into a vulnerable, unparameterized delete query, the likely result is:
- A) The query safely fails with an error
 - B) Every row in the table gets deleted, not just one matching record**
 - C) Only the row named "X" gets deleted
 - D) Nothing happens because SQL ignores quotation marks
11. Before performing a Delete operation in a GUI application, best practice recommends:
- A) Deleting immediately without any warning
 - B) Showing a confirmation dialog to the user before deleting data**
 - C) Disabling the database connection permanently
 - D) Automatically deleting all related tables as well
12. `messagebox.askyesno("Confirm Delete", "Delete this record?")` is used to:
- A) Automatically delete the record without asking
 - B) Prompt the user with a Yes/No confirmation dialog before proceeding**
 - C) Insert a new record into the database
 - D) Display all records in a Listbox
13. In a GUI application, after successfully inserting a new record into the database, which function is typically called to update the Listbox with the latest data?
- A) `connection.commit()` only
 - B) A "load" or "refresh" function that re-runs a `SELECT` query and repopulates the**

widget

C) `window.mainloop()`

D) `cursor.close()`

14. `listbox.delete(0, tk.END)` inside a `load_students()` function is used to:
- A) Permanently delete all students from the database
 - B) Clear the Listbox widget's current contents before repopulating it**
 - C) Delete the database connection
 - D) Remove the first item from the database table only
15. Why does a well-designed GUI application typically call its "load/refresh" function immediately after every Create, Update, or Delete operation?
- A) To permanently close the database connection
 - B) To ensure the GUI display stays in sync with the actual current state of the database**
 - C) To automatically generate a new Primary Key
 - D) To prevent the event loop from starting
-

Section B: Short Answer Questions (SQs)

Topic 4.1.1: What is a GUI? Why is it Important?

1. Define the term "Graphical User Interface" in your own words.
2. Differentiate between a Command-Line Interface (CLI) and a Graphical User Interface (GUI).
3. Explain briefly why GUIs are considered more accessible to non-technical users than CLIs.
4. State two real-world categories of software that commonly use GUIs.
5. Briefly describe the historical contribution of Xerox PARC to the development of the modern GUI.
6. Explain the significance of Steve Jobs's 1979 visit to Xerox PARC in the history of computing.
7. What does the acronym WIMP represent, and why is it relevant to GUI history?

Topic 4.1.2: Overview of Tkinter as Python's Built-in GUI Toolkit

1. Define Tkinter and explain its relationship to the Python programming language.
2. List two advantages of using Tkinter for building small-to-medium GUI applications.
3. Explain briefly what the `mainloop()` function does within a Tkinter program.
4. Identify the function used to create the main application window in Tkinter.
5. Predict what would happen if `window.mainloop()` were removed from a working Tkinter program.

6. Differentiate between `window.title()` and `window.geometry()` in terms of what each one controls.

Topic 4.1.3: Tkinter Components and Widgets

1. Define the term "widget" as it applies to Tkinter.
2. Differentiate between a Frame and a Widget in Tkinter.
3. Explain briefly the purpose of dividing a window into multiple Frames.
4. List the five common Tkinter widgets discussed in this chapter.
5. Differentiate between a Label and an Entry widget in terms of user interaction.
6. Explain briefly when you would choose a Listbox over a Menu widget.
7. State the purpose of the `pack()` call used after creating a widget such as `label.pack(pady=10)`.
8. Identify which widget would be most appropriate for allowing a user to select one item from a scrollable list of options.

Topic 4.1.4: Layout Management

1. Define the term "layout manager" in the context of Tkinter.
2. Differentiate between `pack()` and `grid()` as layout management methods.
3. Differentiate between `grid()` and `place()` in terms of how precisely widgets are positioned.
4. Explain briefly why mixing `pack()` and `grid()` within the same parent container causes problems.
5. State two design qualities that make an interface "clean and responsive."
6. Explain briefly what `columnspan=2` accomplishes when used with the `grid()` method.
7. Predict what might go wrong with a layout built using `place()` if the user resizes the window.
8. Explain briefly the role of `grid_columnconfigure(..., weight=1)` in building a responsive layout.

Topic 4.1.5: Event Handling and Interactivity

1. Define "event-driven programming" in your own words.
2. Differentiate between an "event" and a "callback function."
3. Explain briefly, using the waiter-and-bell analogy or a similar example, how the Tkinter event loop works.
4. Differentiate between using `command=` and using `.bind()` to connect a widget to a function.
5. Identify the mistake in the following code and explain why it is incorrect: `button = tk.Button(window, text="Submit", command=my_function())`.

6. Explain briefly why a Tkinter window freezes and becomes "Not Responding" when a long-running loop is placed inside a button's callback function.
7. State the purpose of the `entry.get()` method in a Tkinter form.
8. Explain briefly the purpose of using `show="*"` when creating a password Entry widget.
9. Map out, in a short step-by-step list, what happens from the moment a user clicks a "Login" button to the moment a success or error message appears.

Topic 4.2.1: Introduction to Databases

1. Define the term "database" in your own words.
2. Differentiate between a flat file (such as a `.txt` file) and a relational database.
3. Define the term "entity" and provide one original example not used in the chapter.
4. Define the term "attribute" and provide one original example not used in the chapter.
5. Differentiate between a "relationship" and an "identifier" in database terminology.
6. Explain briefly the historical contribution of Edgar F. Codd to modern computing.
7. State one reason why using a database is generally more reliable than storing data in a plain text file.

Topic 4.2.2: Overview of Relational Databases and SQL Structure

1. Define "SQL" and state its full form.
2. Differentiate between a "row" (record) and a "column" (field) in a database table.
3. Differentiate between a Primary Key and a Foreign Key.
4. Explain briefly why every table should have a Primary Key.
5. Identify, using a short example of your own, a scenario where a Foreign Key would be needed to connect two tables.
6. Explain briefly why a column with repeating values cannot be used as a Primary Key.

Topic 4.2.3: Connecting Python to a Database

1. Define the term "cursor" as it applies to Python's `sqlite3` library.
2. Differentiate between `sqlite3` and MySQL in terms of typical use cases.
3. Explain briefly, using the bank vault analogy or your own analogy, the purpose of the `connect()` function.
4. State the SQL command syntax used to create a table only if it does not already exist.
5. Explain briefly the purpose of the `commit()` method, and what happens if it is omitted.
6. Identify the method used to retrieve all rows from a query result as a Python list.
7. Map out the correct order of steps needed to connect to a database, insert a record, and permanently save it.

Topic 4.2.4: CRUD Operations in Python GUI Applications

1. Define the acronym CRUD and state the SQL command associated with each letter.
 2. Differentiate between the "Read" operation and the other three CRUD operations in terms of how each affects stored data.
 3. Explain briefly what a "parameterized query" is and why it is considered safer than string concatenation.
 4. Explain briefly what SQL injection is, using the `' OR '1'='1'` example.
 5. Identify why using `messagebox.askyesno()` before a Delete operation is considered good practice.
 6. Predict what would happen to a `students` table if the query `"DELETE FROM students WHERE name = '" + name + "'"` were executed with `name` set to `X' OR '1'='1'`.
 7. Explain briefly why a GUI application should call a "load" or "refresh" function immediately after every Create, Update, or Delete operation.
 8. Map out, in a short step-by-step list, how clicking an "Add Student" button leads to a new row appearing in both the database and the Listbox.
-

Section C: Long / Extensive Questions (LQs)

Topic 4.1: Graphical User Interface (GUI) Development with Tkinter

1. Discuss in detail the concept of a Graphical User Interface and its overall importance in modern application development.
 - a) Define GUI and explain how it differs fundamentally from a Command-Line Interface.
 - b) Discuss the historical development of the GUI, referencing Xerox PARC, Alan Kay, and Steve Jobs's role in popularizing it.
 - c) Evaluate the advantages and potential limitations of GUI-based applications compared to CLI-based applications for different types of users.
2. Elaborate on Tkinter as Python's built-in GUI toolkit and construct a complete explanation of how a basic Tkinter application is built from start to finish.
 - a) Explain the role of `tk.Tk()`, widget creation, layout placement, and `mainloop()` in the lifecycle of a Tkinter application.
 - b) Construct a fully working example (in pseudocode or Python) of a window containing a Label, an Entry field, and a Button.
 - c) Analyze what would go wrong, step by step, if `mainloop()` were omitted from your example.

3. Analyze the key components and widgets available in Tkinter, and discuss how Frames contribute to organizing a complex interface.
 - a) Discuss the role of each of the following widgets: Label, Button, Entry, Menu, and Listbox, giving a real-world software example for each.
 - b) Explain, with a constructed example, how a window can be divided into a top Frame and a bottom Frame, each containing different widgets.
 - c) Evaluate why grouping widgets into Frames improves the maintainability of a large-scale GUI application.
 4. Discuss in detail the three layout management methods available in Tkinter — `pack()`, `grid()`, and `place()`.
 - a) Explain, with examples, how each layout manager positions widgets differently.
 - b) Construct a comparison table evaluating `pack()`, `grid()`, and `place()` across the criteria of: ease of use, precision of control, and responsiveness to window resizing.
 - c) Design a registration form layout (on paper, using an ASCII wireframe or description) using `grid()`, explaining your row and column choices.
 - d) Justify why mixing `pack()` and `grid()` in the same parent container is considered poor practice, and propose how a developer could still use both layout managers within one window correctly.
 5. Analyze the concept of event-driven programming as implemented in Tkinter, and discuss its overall significance for interactive applications.
 - a) Explain the event loop cycle in detail, from `mainloop()` starting to a callback function executing and the loop returning to an idle state.
 - b) Differentiate between `command=` and `.bind()`, providing an example scenario where each would be the appropriate choice.
 - c) Trace, step by step, what happens internally and on-screen when a user types a username and password and clicks "Login" in a simple login form, including both the success and failure paths.
 - d) Evaluate the consequences of placing long-running code directly inside an event callback function, and propose a strategy a developer might use to avoid freezing the GUI.
-

Topic 4.2: Working with Databases in Python

1. Discuss in detail the transition from flat-file data storage to relational databases, and analyze why relational databases became the industry standard.
 - a) Explain the limitations of flat files (such as `.txt` or `.csv` files) for storing structured, related data.
 - b) Discuss Edgar F. Codd's contribution to the relational database model and explain why his

approach solved the problems of earlier data storage methods.

c) Define and differentiate between the concepts of Entity, Attribute, Relationship, and Identifier, using an original example of your own (not Student/Department).

2. Analyze the structure of relational databases and SQL, and construct a complete schema design exercise.

a) Explain the roles of Tables, Rows (Records), Columns (Fields), Primary Keys, and Foreign Keys in a relational database.

b) Design two related tables (other than Student/Department) — specify the columns, data types, Primary Key, and Foreign Key for each — and justify your design choices.

c) Discuss what would go wrong if the Foreign Key in your second table did not correctly correspond to a Primary Key value in your first table.

3. Discuss in detail the process of connecting a Python program to an SQLite database and executing SQL commands from within Python.

a) Explain, step by step, the purpose of `connect()`, `cursor()`, `execute()`, `commit()`, and `close()`.

b) Construct a complete trace table showing the database's internal state after each line of a program that connects to a database, creates a table, inserts two records, and reads them back.

c) Evaluate the consequences of forgetting to call `commit()` after performing an `INSERT` operation, and explain why this is a common mistake for beginners.

4. Analyze and evaluate the four CRUD operations (Create, Read, Update, Delete) as they apply to a Python GUI application connected to a database.

a) Explain the SQL command and typical GUI trigger (e.g., button click) associated with each CRUD operation.

b) Construct a complete GUI-to-database trace table for a "Student Manager" application, showing how clicking "Add Student" leads to a database change and a Listbox update.

c) Discuss why the "Read" operation is considered fundamentally different from the other three CRUD operations in terms of its effect on stored data.

5. Discuss in detail the security risks of unsafe SQL query construction, using the historical case of SQL injection as a foundation for your analysis.

a) Explain the discovery of SQL injection in 1998 by the researcher known as Rain Forest Puppy, and describe the underlying vulnerability that made it possible.

b) Construct an example of a vulnerable Python function that deletes a record using string concatenation, and trace exactly what happens when a malicious input such as `X' OR '1'='1` is submitted.

c) Justify, with a corrected code example, why parameterized queries (using `?` placeholders) prevent this class of attack.

- d) Evaluate the broader real-world impact of SQL injection vulnerabilities on organizations that manage sensitive user data (e.g., login credentials, financial records), and discuss what ethical responsibilities a software developer has when handling user input.
6. Design and construct a complete end-to-end GUI application that integrates Tkinter with an SQLite database to perform all four CRUD operations.
- a) Design the GUI layout (widgets and layout manager) needed to Create, Read, Update, and Delete student records.
 - b) Construct the corresponding Python functions for each CRUD operation, ensuring all database-modifying operations use parameterized queries.
 - c) Trace, step by step, what happens across the GUI and the database when a user adds a new student, updates that student's grade, and then deletes the record — include the state of the Listbox widget at each stage.
 - d) Evaluate one potential real-world consequence of omitting a confirmation dialog before the Delete operation in your design.